

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ — ĐHQG HÀ NỘI (UET)

Báo cáo Bài tập lớn môn Lập trình Hướng đối tượng (OOP)



HỆ THỐNG ĐẤU GIÁ REAL-TIME

Auction System — Client-Server Desktop Application

THÀNH VIÊN NHÓM 7

Lê Công Xuân Phúc — 25023358 ★ **Nhóm trưởng**

Hồ Anh Tuấn — 25023384

Nguyễn Anh Tùng — 25023389

Trần Thanh Nhật — 25023348

Tháng 06 / 2026

1. Mục tiêu & Phạm vi thực hiện

1.1. Mục tiêu

Xây dựng một **hệ thống đấu giá trực tuyến real-time** hoạt động theo mô hình Client-Server, cho phép nhiều người dùng đồng thời tham gia đặt giá thầu (bid) các sản phẩm. Hệ thống được thiết kế nhằm áp dụng các nguyên lý **Lập trình Hướng đối tượng (OOP)**, các **Design Patterns** tiêu biểu, và xử lý **đa luồng (Concurrency)** trong môi trường thực tế.

1.2. Phạm vi chức năng



Quản lý người dùng

Đăng ký / Đăng nhập với 3 vai trò: Bidder, Seller, Admin. Mật khẩu mã hóa BCrypt.



Quản lý sản phẩm

CRUD sản phẩm đấu giá: Electronics, Art, Vehicle. Hỗ trợ phân trang (Pagination).



Đấu giá Real-time

Đặt giá thầu trực tuyến, cập nhật real-time qua socket TCP/IP. Hỗ trợ Auto-bidding.



Anti-Sniping

Cơ chế chống thầu chụp giờ cuối — tự động gia hạn 20 giây khi có bid trong 10 giây cuối.



Thanh toán & Số dư

Quản lý balance của Bidder, nạp tiền (top-up), thanh toán & hủy đơn sau khi thắng giá.



Thông báo & Lịch sử

Thông báo real-time cho người theo dõi (Observer Pattern). Lưu lịch sử giao dịch.



Analytics & Thống kê

Phân tích số lượng bid, giá trung bình, xu hướng giá cho từng sản phẩm đấu giá.



Admin Dashboard

Quản trị viên có giao diện riêng để quản lý toàn bộ người dùng, xóa tài khoản vi phạm.

1.3. Công nghệ sử dụng

Java 17+

JavaFX 17

TCP/IP Socket

SQLite + JDBC

BCrypt

Maven

Jackson JSON

JUnit 5

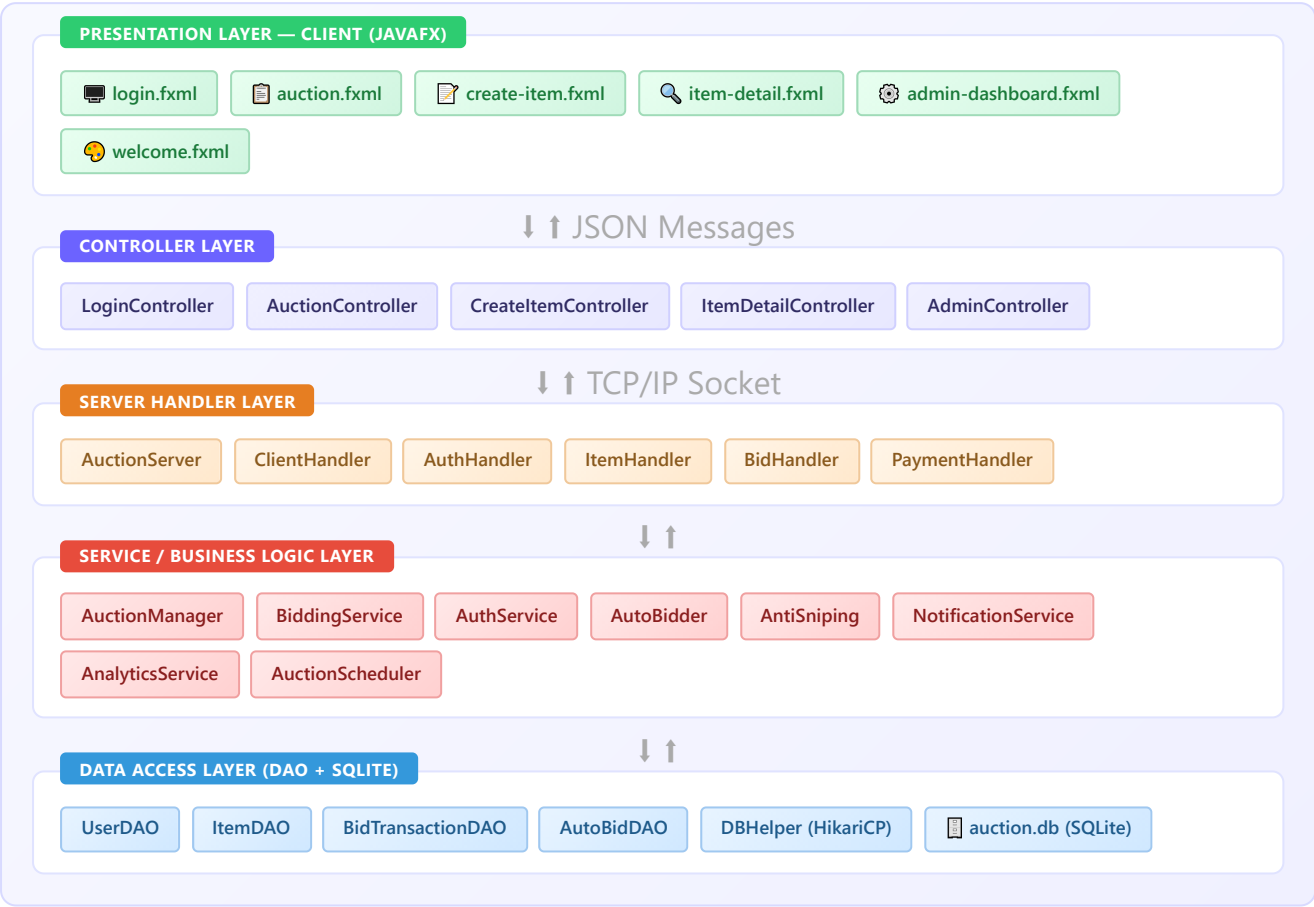
SLF4J + Logback

Checkstyle

GitHub Actions CI

2. Kiến trúc tổng thể hệ thống

Hệ thống được xây dựng theo mô hình **Client-Server đa tầng (Multi-layered Architecture)**, kết hợp các design pattern để đảm bảo tính mở rộng, bảo trì và tái sử dụng mã nguồn. Kiến trúc được chia thành **5 tầng chính**:



Mô tả các tầng kiến trúc

- Presentation Layer:** Giao diện JavaFX + FXML, giao tiếp với Server qua `ClientConnection` (TCP socket). Dữ liệu trao đổi ở định dạng JSON (Jackson).
- Controller Layer:** Nhận sự kiện từ UI, gửi Message tới Server và xử lý Response trả về từ Server.
- Server Handler Layer:** `AuctionServer` lắng nghe kết nối, phân luồng bằng `ExecutorService`, điều hướng message tới Handler tương ứng. Hỗ trợ broadcast thông báo tới tất cả client.
- Service Layer:** Xử lý toàn bộ business logic — đấu giá, xác thực, auto-bid, anti-sniping, scheduling, analytics.
- Data Access Layer:** Các DAO truy xuất SQLite thông qua JDBC, đảm bảo tách biệt giữa logic nghiệp vụ và truy vấn dữ liệu.

3. Chức năng & Design Patterns

3.1. Áp dụng OOP — 4 trụ cột

Encapsulation (Đóng gói)

Tất cả thuộc tính khai báo `private`, truy cập qua getter/setter. Đã refactor toàn bộ từ `protected` sang `private` để đảm bảo tính đóng gói.

Inheritance (Kế thừa)

`Entity` → `User` / `Item` (abstract). `User` có 3 lớp con: Bidder, Seller, Admin. `Item` có 3 lớp con: Electronics, Art, Vehicle.

Polymorphism (Đa hình)

Phương thức `printInfo()`, `displayProfile()` được override ở mỗi lớp con. `BiddingStrategy` interface cho phép thay đổi chiến lược đấu giá linh hoạt.

Abstraction (Trừu tượng)

Abstract class `Item`, `User` định nghĩa hành vi chung. Interface `BidObserver`, `BiddingStrategy` trừu tượng hóa hành vi hệ thống.

3.2. Design Patterns đã áp dụng

Factory Pattern — `ItemFactory`

Vấn đề: Cần tạo các loại sản phẩm khác nhau (Electronics, Art, Vehicle) với thuộc tính riêng biệt.

Giải pháp: `ItemFactory.createItem(type, ...)` nhận tham số `type` và trả về đối tượng Item tương ứng mà không cần client biết chi tiết lớp con.

Lý do: Tăng tính mở rộng — khi thêm loại sản phẩm mới chỉ cần thêm case trong Factory, không sửa code client. Tuân thủ Open/Closed Principle.

Observer Pattern — `BidObserver` + `NotificationService`

Vấn đề: Cần thông báo cho tất cả người theo dõi khi có bid mới trên sản phẩm đấu giá.

Giải pháp: Interface `BidObserver` với phương thức `update(itemId, price, bidderId)`. `NotificationService` quản lý danh sách observer và broadcast thông báo. `Bidder` implements `BidObserver`.

Lý do: Giải bài toán publish/subscribe — thêm/xóa observer mà không sửa logic đấu giá.

Singleton Pattern — `AntiSniping` + `RealtimeNotifier`

Vấn đề: Cần đảm bảo chỉ có duy nhất một instance quản lý thời gian đấu giá và thông báo realtime.

Giải pháp: Double-checked locking Singleton trong `AntiSniping.getInstance()` và `RealtimeNotifier.getInstance()`.

Lý do: Tránh xung đột dữ liệu khi nhiều thread cùng truy cập tài nguyên dùng chung (thời gian đấu giá).

Strategy Pattern — `BiddingStrategy`

Vấn đề: Cần linh hoạt thay đổi quy tắc kiểm tra bid hợp lệ (bid cao hơn giá hiện tại, hoặc cao hơn ít nhất 5%).

Giải pháp: Interface `BiddingStrategy` với method `isValidBid(currentHighest, newBid)`. `DefaultBiddingStrategy` kiểm tra `newBid > currentHighest`.

Lý do: Có thể thêm strategy mới (MinIncrementStrategy, PercentageStrategy) mà không sửa logic hiện tại — tuân thủ Open/Closed Principle.

Template Method — `Item.updateHighestBid()`

Vấn đề: Các loại Item có cùng thuật toán cập nhật giá cao nhất nhưng có thể mở rộng thêm logic riêng.

Giải pháp: Phương thức `updateHighestBid()` trong abstract class `Item` định nghĩa skeleton algorithm, các lớp con override nếu cần.

Lý do: Đảm bảo nhất quán trong logic cập nhật giá, đồng thời cho phép tùy biến ở lớp con.

4. Chức năng nâng cao & Concurrent

4.1. Xử lý đa luồng (Concurrency)

Hệ thống xử lý nhiều client đồng thời thông qua **thread pool** với `ExecutorService` (`CachedThreadPool`). Các cấu trúc dữ liệu thread-safe được sử dụng xuyên suốt:

Thread-safe Collections

- `ConcurrentHashMap` cho `activeAuctions`, `bidders`, `auctionEndTime`
- `CopyOnWriteArrayList` cho `observers`, `transactionHistory`, `clients`

Concurrent Mechanisms

- `ScheduledExecutorService` cho `AuctionScheduler` & countdown timer
- `synchronized` double-checked locking cho Singleton

4.2. Auto-Bidding (Đấu giá tự động)

`AutoBidder` cho phép người dùng đặt giá trần (max bid). Khi có bid mới từ người khác, hệ thống tự động đặt bid cao hơn cho đến khi đạt giới hạn. Auto-bid được lưu trong database (`AutoBidDAO`) và được kích hoạt tự động phía server.

4.3. Anti-Sniping (Chống thầu chụp giờ cuối)

Khi phát hiện bid mới trong **10 giây cuối cùng** của phiên đấu giá, hệ thống tự động **gia hạn thêm 20 giây**. Cơ chế này ngăn chặn chiến thuật "sniping" — đặt bid vào giây cuối để người khác không kịp phản hồi. Implementation sử dụng Singleton `AntiSniping` với `ConcurrentHashMap` lưu `endTime` cho từng phiên.

4.4. Real-time Communication

Server duy trì danh sách `CopyOnWriteArrayList<ClientHandler>` để **broadcast** message tới tất cả client khi có sự kiện: bid mới, item được tạo/xóa/cập nhật, thanh toán thành công. Message được serialize/deserialize bằng Jackson `ObjectMapper`.

5. Xử lý lỗi & Bảo mật

Exception Handling

- `InvalidBidException` — giá bid thấp hơn giá hiện tại
- `AuctionClosedException` — phiên đã kết thúc
- `AuthenticationException` — sai mật khẩu / user không tồn tại
- `AuctionException` — exception tổng quát
- Xử lý `SQLException` trong tất cả DAO

Bảo mật

- BCrypt hashing** — mật khẩu không lưu plain text
- UUID** — ID người dùng ngẫu nhiên, không dự đoán được
- Input validation** — kiểm tra giá bid, quyền truy cập
- Role-based access** — phân quyền Bidder/Seller/Admin

6. Kiểm thử & CI/CD

Unit Testing (JUnit 5)

- `AuctionManagerTest` — quản lý đấu giá, đặt bid, bid không hợp lệ
- `AuthServiceTest` — đăng ký, đăng nhập, xác thực BCrypt
- `DefaultBiddingStrategyTest` — kiểm thử chiến lược bid

CI/CD & Code Quality

- GitHub Actions** — tự động chạy `mvn test` khi push/PR
- Checkstyle** — kiểm tra coding conventions tự động
- Maven Build** — quản lý dependencies và packaging
- Git** — version control với branching strategy

7. Phân chia công việc

Thành viên	MSSV	Công việc chính	Package / Module
Lê Công Xuân Phúc (Nhóm trưởng)	25023358	- Thiết kế kiến trúc tổng thể Client-Server - Xây dựng Server, Handler Layer - Implement AuctionServer, ClientHandler - Xử lý broadcast & real-time communication - Tích hợp CI/CD (GitHub Actions)	serverhandlerclient .github/workflows
Hồ Anh Tuấn	25023384	- Xây dựng Entity layer (User, Item hierarchy) - Implement Factory Pattern (ItemFactory) - Implement Observer Pattern (BidObserver) - Xây dựng DAO layer (UserDAO, ItemDAO) - Thiết kế database schema (SQLite)	entityfactorydao util
Nguyễn Anh Tùng	25023389	- Xây dựng Service Layer (BiddingService, AuthService) - Implement Strategy Pattern, Singleton Pattern - Implement Auto-Bidding & Anti-Sniping - Xử lý Concurrency (thread-safe collections) - Viết Unit Tests (JUnit 5)	service/auction service/bidding service/authtest
Trần Thanh Nhật	25023348	- Thiết kế giao diện JavaFX (FXML + CSS) - Xây dựng Controller layer - Implement Notification Service, Analytics - Xử lý thanh toán (PaymentHandler) - Viết tài liệu báo cáo & README	controller resources (FXML) service/notification service/analytics

Tổng kết barem điểm đạt được

- ✓ **OOP** — Inheritance, Polymorphism, Encapsulation, Abstraction
 - ✓ **Design Patterns** — Singleton, Factory, Observer, Strategy, Template Method
 - ✓ **Concurrency** — Thread-safe collections, ExecutorService, ScheduledExecutor
- ✓ **Exception Handling** — Custom exceptions cho từng loại lỗi
 - ✓ **Unit Tests** — JUnit 5 cho core services
 - ✓ **Build & CI/CD** — Maven, Checkstyle, GitHub Actions

Điểm nổi bật của dự án

- Real-time bidding** — đấu giá trực tiếp qua TCP socket
 - Auto-Bidding** — đặt giá tự động theo giá trần
 - Anti-Sniping** — chống thầu chụp giờ cuối
- Multi-role system** — Bidder, Seller, Admin
 - BCrypt security** — mã hóa mật khẩu an toàn
 - Full-stack desktop app** — JavaFX + Server + SQLite